Test Strategy — QA Hackathon (Automation Exercise)

Target Application: https://automationexercise.com **Repository artefacts:** testcases.csv · defects.csv · automation/tests/

---

1. Functional Test Strategy

1.1 Scope & Modules Covered

40 test cases were authored across 11 modules:

Module	Cases	Coverage focus
Home	TC001–TC005	Page load, navigation, newsletter subscription, scroll control
Signup	TC006–TC009, TC040	Happy path, duplicate email, empty/invalid input, boundary
Login	TC010–TC014	Valid + invalid + empty + logout
Account	TC015	Account deletion
Products	TC016–TC021	Listing, detail, search valid/invalid/empty/special-chars
Cart	TC022–TC027	Add, view, update qty, remove, empty state, qty merge
Checkout	TC028–TC031	Login gate, address review, place order, payment validation
Contact	TC032–TC033	Form submission with file upload + validation
Informational	TC034–TC035	Test Cases and API Testing reference pages
UI	TC036–TC037	Responsive layout, broken-image scan
Security	TC038–TC039	Reflected XSS in search, SQLi attempt on login

1.2 Risk-Based Prioritization

- **High (must work for any release):** signup happy path, duplicate-email guard, login (valid + invalid + empty + logout), account deletion, all-products listing, product detail, search returning results, cart add/view/update/remove, checkout login gate, checkout review, place order with valid card, payment field validation, XSS, SQLi.
- **Medium:** newsletter subscription, search no-result, special-character search, cart-empty state, qty merge, contact form happy path and validation, responsive layout, broken-image scan.
- **Low:** scroll-up, informational pages, signup boundary input.

1.3 Test Types Mix

- **Functional** (happy paths) — ~55% — proves end-to-end flows work for the primary persona.
- **Negative** (~25%) — invalid creds, duplicate signup, invalid coupon-style inputs, empty fields, unknown email.
- **Boundary** (~5%) — very long name on signup.
- **UI** (~10%) — responsive viewport, broken-image detection, navigation, scroll control.
- **Security** (~5%) — XSS payload in search, SQLi payload in login.

1.4 Coverage Gaps & Out-of-Scope (this iteration)

- **Payment-failure paths** — only happy-path payment + empty-payment are exercised; declined card / 3DS / expired card not covered (the demo has no real gateway).
- **Email verification** — order confirmation / signup emails not asserted (requires mail-sink such as Mailpit).
- **Concurrency / cart-race conditions** — no parallel-session tests.
- **Performance / load** — out of scope; recommend a separate k6/Locust suite.
- **Accessibility (WCAG)** — no axe-core scan yet; flagged BUG002 manually (hover-only Add to Cart).
- **Cross-browser** — only Chromium configured; Firefox/WebKit projects can be added trivially.
- **Authenticated-cart persistence** — verifying that cart contents survive login/logout cycles was not exercised.

1.5 Recommendations

1. Add `@axe-core/playwright` and run an accessibility scan on home, products, product detail, cart, checkout and contact.
2. Seed a deterministic test user via API (POST `/createAccount`) so login tests do not depend on a fresh signup every run (faster and more reliable).

3. Add a mail-sink in CI (Mailpit / GreenMail) to assert signup-confirmation and order-confirmation emails.
4. Extend payment tests with declined / invalid card numbers once a sandbox gateway is wired up.
5. Add cross-browser projects (Firefox, WebKit) to expose engine-specific issues.

---

2. Automation Test Strategy

2.1 Framework Rationale — Playwright + TypeScript + MCP

- **Playwright** provides auto-waiting, role-based locators, file-upload, video/trace artefacts and parallel workers — the right tool for full-stack e-commerce flows.
- **TypeScript** keeps shared helpers (`signUpNewUser` , `loginUser`) safe and self-documenting.
- **Playwright MCP browser tools** were used during authoring to live-explore the application, discover the exact selectors (`#susbscribe_email` (note the typo in the product), `#search_product` , `#submit_search` , `.shop-menu a[href=...]` , `#cart_info_table` , etc.) and confirm form names before committing locators. This eliminated almost all "fix-on-first-run" iterations for selectors.

2.2 Scope of Automation

Layer	Status
Smoke (home loads, products list)	Automated
Auth (signup, login, logout, delete account)	Automated
Products & search (happy + negative + special chars)	Automated
Cart (add, view, update, remove, qty merge, empty state)	Automated
Checkout (login gate, review, place order, payment validation)	Automated
Contact (form submit with file upload)	Automated
Informational pages (Test Cases, API Testing)	Automated
UI (responsive, broken images)	Automated
Basic security (reflected XSS, SQLi in login)	Automated
Email-based flows	**Manual** (no mail-sink)

Layer	Status
Real payment-gateway responses	**Manual**
Exploratory / UX	**Manual**

40 test functions live in **12 spec files** under automation/tests/ plus a helpers.ts shared module.

2.3 Locator Strategy

- **Preferred (user-facing):** `getByRole`, `getByLabel`, `getByText`, `getByPlaceholder` — survives CSS refactors.
- **Stable IDs and form `name` attributes:** used where role-based selection is ambiguous or duplicated (e.g. `.signup-form input[name="email"]` vs `.login-form input[name="email"]`).
- **Href-based selectors** for navigation: `.shop-menu a[href="/products"]` — chosen because nav text contains leading/trailing whitespace on this site that broke exact-text matching.
- **`.first()` discipline:** the site has several DOM elements that intentionally repeat content (e.g. "Order Placed!" appears twice on the confirmation page); locators are constrained with `.first()` or scoped to a unique parent (`#contact-page`) to avoid Playwright strict-mode violations.
- **Avoided:** brittle nth-child CSS and XPath.

2.4 Test Design Patterns

- **Module-per-file** structure for clear ownership and easy `--grep` filtering.
- **Serial mode** within order-dependent flows (Cart lifecycle) via `test.describe.configure({ mode: 'serial' })`.
- **Data independence:** every signup test generates a unique timestamp-suffixed email — no shared accounts.
- **Robust waits:** explicit `toBeVisible({ timeout })` on user-facing success/failure text instead of `waitForTimeout`.
- **Artefact retention:** HTML + JSON reporters, `screenshot: only-on-failure`, `video: retain-on-failure`, `trace: on-first-retry`.
- **Tests as defect documentation:** TC033 deliberately asserts the *expected* behavior so the test failure surfaces the defect (BUG001). This keeps the defect alive in CI until fixed.

2.5 CI Considerations

- `forbidOnly: true` and `workers: 1` enforced in CI via the `CI` env var pattern in playwright.config.ts.
- `retries: 0` by default — clean signal in this run; bump to `1` only if real flakiness emerges.
- Recommended pipeline (next iteration):
  1. `npm ci` in `automation/`.
  2. `npx playwright install --with-deps chromium`.
  3. `npx playwright test`.
  4. Upload `playwright-report/`, `test-results/`, `testcases.csv`, `defects.csv` as build artefacts.
  5. Optionally add a job that runs the axe-core accessibility scan separately.

2.6 Key Defects Found (Summary)

Full detail in defects.csv.

ID	Severity	Summary
BUG001	Medium	**Contact Us form does not enforce required fields** — Name and Subject inputs lack the `required` attribute; users can submit blank enquiries. TC033 deliberately asserts the desired behavior so the failure tracks the defect.
BUG002	Low	**Hover-only Add-to-cart on product card** — the overlay variant of the "Add to cart" button is invisible to keyboard users and assistive tech, an accessibility concern (WCAG 2.4.7 / 2.1.1).
BUG003	Low	**Console errors on initial page load** — 3–5 JavaScript console errors emitted on every navigation. Functionality unaffected but log pollution masks real issues.
BUG004	Low	**Duplicate `id="success-subscribe"`** on the page — both the home-page subscription banner and the contact-form success banner share the same element id. Invalid HTML and a real automation hazard (caused multiple strict-mode locator collisions).

2.7 Run Summary

- 40 tests total · **39 passed** · **1 intentional failure** (TC033 — gates BUG001 fix).
- Wall-clock: ~5 minutes single-worker.

- No environmental blockers (target site is open and stable, unlike a prior run against a Cloudflare-fronted demo).

2.8 Lessons Learned

- The **MCP-driven exploration phase paid off** — discovering things like the misspelled `#susbscribe_email`, the leading whitespace in nav text, and the form `name`-attribute differences between the signup/login twin forms before writing any spec saved an entire fix-on-first-run cycle.
- **Strict mode is your friend** — every multi-match error from Playwright pointed at a real DOM design smell (duplicate ids, duplicate success banners). It's worth resisting the urge to slap `.first()` on every locator and instead scope by a unique parent.
- **Hover-only controls are a recurring anti-pattern** in e-commerce demos; worth a dedicated accessibility checklist item in the strategy.

Generated for the QA Hackathon (Automation Exercise dry-run) — Phases 1, 3 and 4. Phase 2 (manual test-case review) is the team's responsibility.